

Start Here

A reading guide for learning from this handbook. If you want to *learn* from the material, read this first. If you want to *contribute* to it, read `CLAUDE.md` and `meta/00-CONSTITUTION.md` instead.

What you need before you start

You should already be able to write basic programs in some language — declare variables, write functions, call an API, run a query. That is the only prerequisite. You do **not** need to know architecture, system design, or any of the specific frameworks used in the examples; the handbook teaches the reasoning, and the code is there to make the reasoning concrete.

If you can read Python and a little TypeScript, you will follow every example. If you can't yet, you can still read for the judgment — the prose explains the "why" independently of the code.

What this handbook is trying to do

It teaches **engineering judgment**, not syntax. AI can now write code faster than you can read it, so the scarce skill is no longer typing — it is deciding what to build, how to structure it, what it will cost over time, and whether the code an assistant just produced is actually correct. Every chapter is built around that shift.

Concretely, that means the goal of reading a chapter is not to memorize an API. It is to be able to answer, for its topic: *why does this exist, when do I use it, when do I not, what does it trade away, and what do people (and AIs) get wrong about it?* If you finish a chapter able to answer those, you've learned it — even if you couldn't write the code from memory.

How the material is organized

The handbook is a **progressive curriculum** of stages, and stages build on each other:

mindset → architecture → building → shipping → scaling → leading
(Stage 1) (Stage 2) (Stages 3–7) (Stage 7) (Stage 11) (Stage 13)

Read the stages **in order the first time**. Later stages assume the mental models from earlier ones — Stage 3 (backend) uses the architecture from Stage 2, which uses the judgment from Stage 1. After the first pass, treat any chapter as a standalone reference you return to.

Availability, right now:

Stage	What you'll be able to do after it	Status
1 — Engineering Mindset	Think about cost, trade-offs, and product value before writing code	Ready
2 — Software Architecture	Choose and justify how to structure a system before building it	Ready
3 — Backend Engineering	Build a production API: persistence, auth, jobs, caching, observability	Ready
4 — Frontend Engineering	Build the Next.js/React frontend: components, data fetching, state, forms, performance	Ready
5 — Mobile Engineering	Ship a React Native app: navigation, auth, offline, notifications, releases	Ready
6 — Database Engineering	Design, index, and evolve a production PostgreSQL schema	Ready
7 — DevOps	Containerize, deploy, and operate the system on real infrastructure with CI/CD	Ready

Stage	What you'll be able to do after it	Status
8 — Testing	Build a trustworthy test suite: strategy, unit, mocking, integration, E2E	Ready
9 — Security	Attack and harden the system: JWT, OAuth, secrets, injection, XSS, CSRF/CORS, rate limiting	Ready
10 — AI Engineering	Engineer the AI collaboration itself: configure agents, context, prompts, multi-agent patterns, review, debugging, and team workflows	Ready
11 — System Design	Scale the system when load grows: find bottlenecks, load balance, scale out, Redis, queues, CDN, event streaming	Ready
12+ (SaaS, Leadership, Case Studies)	The rest of the senior-engineer path	In progress

You do not need to wait for the rest. Stages 1–11 take you from mindset to a built, deployed, tested, secured, and scalable production system — engineered with AI as a disciplined teammate. Start now; new stages become readable the moment they land.

How every chapter is structured — and how to use each part

Every chapter follows the same shape, so once you know the rhythm you can read efficiently. The sections are not all meant to be read the same way:

Read these first, carefully — they carry the learning:

- **Introduction / Why It Matters** — what the topic is and what it costs to get wrong. Sets the stakes.
- **Mental Model** — the one abstraction to carry out of the chapter, usually with a diagram and a one-sentence definition. If you remember one thing from a chapter, remember this.
- **Real-World Scenario** (mindset chapters) or **Production Example** → **Folder Structure** → **Implementation** (technical chapters) — the concept made concrete in a realistic situation or real code. This is where it clicks.
- **Engineering Decisions** — the actual choices, with the options that were rejected and why. This is the judgment, shown explicitly. Read it slowly.

Read these to sharpen judgment:

- **Trade-offs** — what the approach costs and when *not* to use it. The counterweight to every recommendation.
- **Common Mistakes / Anti-Patterns** — how it goes wrong, so you recognize it.
- **AI Mistakes** — the specific ways Claude Code, GPT-based tools, and Cursor get this topic wrong, each with how to detect it and how to fix it. **This is the most AI-first part of the handbook** — read it as a review checklist for the next time an assistant writes this kind of code for you.

Use these as reference, not front-to-back reading:

- **Decision Tree** — a compressed "if X do A" you can consult during a real design discussion.
- **Checklist** — implementation / architecture / code-review (and sometimes deployment) checklists to run against actual work.
- **Exercises** — do these (see below).
- **Further Reading** — where to go deeper on that specific topic.

The running example: Invoicely

One fictional product runs through the entire handbook: **Invoicely**, a B2B invoicing SaaS (FastAPI + PostgreSQL backend, Next.js frontend). You watch the same system get designed, built, and evolved across chapters — its billing, its reconciliation engine, its multi-tenant data, its background jobs. Following one product instead of disconnected snippets is deliberate: it lets you see how decisions in one chapter constrain and enable decisions in the next, which is what real engineering feels like.

How to actually learn from it (not just read it)

Passive reading of an engineering handbook produces the illusion of understanding. Three habits turn it into real judgment:

1. **Do the exercises.** Every chapter ends with 2–3 exercises that *produce an artifact* — a written analysis, a refactored diff, a decision you defend, a bug report. They resemble real engineering work on purpose. Doing even one per chapter is worth more than reading three extra chapters.
2. **Use the AI Mistakes sections as a live review tool.** The next time an AI assistant writes code for a topic you've read, pull up that chapter's AI Mistakes section and check the generated code against it. This is where the handbook pays for itself immediately — it turns you into a better reviewer of AI output, which is the core AI-first skill.
3. **Apply the reusable assets to real work.** The handbook ships more than prose:
 - **templates/** — copy-and-fill documents: an ADR, a pull-request description, a project brief, and a `CLAUDE.md` starter for your own AI-first repositories.
 - **checklists/** — a code-review checklist (with a dedicated AI-generated-code section) and a production-readiness checklist.
 - **playbooks/** — end-to-end processes, starting with *Starting an AI-First Project*, which ties the templates and checklists together into a workflow you can follow on a real project today.

Reading it alongside an AI assistant

This is an AI-first handbook, so read it the way it's meant to be used — with an assistant, not instead of one:

- After a chapter, ask your assistant to build the chapter's example, then review its output against the chapter's **Checklist** and **AI Mistakes** sections. You'll catch things you'd have missed a week ago.
- Drop the chapter's principles into your project's `CLAUDE.md` (there's a starter template) so the assistant follows them by default.
- Treat the **Decision Trees** as prompts you can hand to an assistant to structure a real design conversation.

The handbook's whole thesis is that the engineer owns the decisions and the AI accelerates the implementation. Reading it this way practices exactly that division of labor.

A suggested first path

1. Read **Stage 1** end to end — it's short, and it reframes everything after it. Do the exercise in Chapter 01 (the "question list") and Chapter 03 (the cold-start test).
2. Read **Stage 2**, and as you go, sketch how you'd structure a system you actually work on. Use the Decision Tree in each chapter.
3. Read **Stage 3** with an assistant open — build pieces of the Invoicely backend (or your own) as you read, and review every piece against the chapter's AI Mistakes section.
4. From there, follow new stages as they land, or jump to whichever stage matches what you're building right now.

Start with [Stage 1, Chapter 01 — What is Software Engineering?](#).